



รายงานผลตรวจคุณภาพโค้ด

PaynEat POS — Clean Code · State Management · Clean Architecture ·
Technical Debt · โครงสร้างโฟลเดอร์

ตรวจทั้งโปรเจกต์ (Flutter app + Node.js backend) 5 มิติ พบ 2 จุดที่ผิดกฎ Clean Architecture และ
1 จุดที่โครงสร้างไม่สม่ำเสมอ — แก้ไขและยืนยันด้วยเทสต์ทั้งหมดแล้วในรายงานนี้

82

เทสต์อัตโนมัติผ่านทั้งหมด

9 / 9

Flutter feature ตรงมาตรฐานเดียวกัน

9 / 9

Backend module ผ่าน layering เดียวกัน

3

บั๊ก/ข้อผิดพลาดจริงที่พบและแก้แล้ว

PaynEat POS · Code Quality Audit

7 กันยายน 2026

ผลตรวจ 5 มิติ

ตรวจด้วยเครื่องมือจริง (flutter analyze, dart format, flutter test, npm test) ร่วมกับการอ่านโค้ดหา dependency violation ทีละไฟล์ ไม่ใช้การประเมินแบบผิวเผิน — ทุกข้อในรายงานนี้อ้างอิงตำแหน่งไฟล์จริงในโปรเจกต์

- 1 Clean Code** ผ่าน
 flutter analyze 0 ปัญหา · dart format สะอาด 142 ไฟล์ · ไม่มี print()/TODO ค้าง · พบไฟล์ใหญ่เกิน 1 ไฟล์และ backend ไม่มี ESLint (บันทึกเป็น debt)
- 2 State Management (GetX)** ผ่าน
 แยก ephemeral state (setState) กับ app state (Rx) ถูกต้องทุกจุด · controller lifecycle ครบ · พบบัก Obx() 1 จุดระหว่างพัฒนาก่อนหน้านี้ — แก้แล้ว
- 3 Clean Architecture** พบ 2 จุด — แก้แล้ว
 Backend layering (routes → controller → service → repository) ไม่มี violation เลยใน 9 module · Flutter domain layer เคย import จาก data layer 2 จุด — ย้ายแล้ว ยืนยันด้วยเทสดีครส
- 4 Technical Debt** ติดตาม 5 รายการ
 2 รายการแก้แล้วระหว่างการตรวจนี้ · 5 รายการยังค้าง (ส่วนใหญ่คือช่องว่างของเกสต์ต่อ controller/module) มีแผนแก้ไขชัดเจนทุกข้อ
- 5 โครงสร้างโฟลเดอร์** พบ 1 จุด — แก้แล้ว
 Flutter 9 feature ตามโครงสร้าง feature-first เหมือนกันหมด · backend เคยมี 3 module ข้าม controller layer — เพิ่มครบแล้ว ตอนนี้ 9 module มีโครงสร้างเดียวกันทั้งหมด

 ผลตรวจฉบับเต็มพร้อมกฎที่บังคับใช้จากนี้ไปถูกบันทึกเป็นเอกสารมีชีวิต (living document) ที่ [docs/CODING_STANDARDS.md](#) ในโปรเจกต์ — อัปเดตทุกครั้งที่คุณพบหรือแก้ปัญหาใหม่ รายงาน PDF นี้คือภาพนิ่ง ณ วันที่ตรวจ

ตรวจสอบด้วยเครื่องมือจริงของแต่ละภาษา ไม่ใช่การอ่านผ่านสายตาอย่างเดียว

ผลการตรวจอัตโนมัติ

เครื่องมือ	ผลลัพธ์	สถานะ
flutter analyze	No issues found!	ผ่าน
dart format --set-exit-if-changed	Formatted 142 files (0 changed)	ผ่าน
flutter test	46/46 เทสต์ผ่าน	ผ่าน
npm test (backend)	36/36 เทสต์ผ่าน	ผ่าน
ค้นหา TODO/FIXME/HACK	ไม่พบทั้งสองฝั่ง	ผ่าน
ค้นหา print() ใน Dart	ไม่พบเลยทั้ง 130 ไฟล์	ผ่าน
ค้นหา console.log ใน backend	พบ 6 จุด — อยู่ใน entry point/error handler ที่กำหนดไว้เท่านั้น ไม่มีใน business logic	ผ่าน

สิ่งที่ต้องปรับปรุง

รายการ	รายละเอียด
ไฟล์ใหญ่เกินไป ควรแยก	core/demo/demo_store.dart — 1,142 บรรทัดในไฟล์เดียว เป็นเซิร์ฟเวอร์จำลองทั้งร้านสำหรับ Demo Mode ยอมรับได้ว่าใหญ่เพราะจำลองทุกโมดูล แต่ถ้าเพิ่ม scenario ใหม่ควรแยกเป็นไฟล์ย่อยตามโดเมน (เช่น demo_store_orders.dart) แทนการเพิ่มในไฟล์เดิม
Backend ไม่มี linter ค้าง	ไม่มี ESLint/Prettier config — พังเทสต์ (36 เทสต์) กับ code review เป็นตัวจับความผิดพลาดเท่านั้น error ทาง syntax/style เล็กๆ (unused var, inconsistent quotes) จะไม่ถูกจับอัตโนมัติ

ขนาดไฟล์ที่ใหญ่ที่สุด (อ้างอิง)

Flutter (บรรทัด)	จำนวน	Backend (บรรทัด)	จำนวน
demo_store.dart	1,142	order.service.js	331
menu_form_page.dart	531	db/seed.js	226
demo_data_sources.dart	453	order.repository.js	218
dashboard_page.dart	394	menu.repository.js	190

ไม่มีเกณฑ์บังคับตายตัว แต่ไฟล์ที่เกิน ~400 บรรทัดควรตั้งคำถามว่าแยกความรับผิดชอบได้ไหม ฟังก์ backend ทุกไฟล์อยู่ในเกณฑ์ที่เหมาะสม ไม่มีไฟล์ไหนเกิน 350 บรรทัด

State Management (GetX)

จุดที่ตรวจเข้มที่สุดในหัวข้อนี้คือการแยก "ephemeral UI state" ออกจาก "app/business state" เพราะเป็นจุดที่มักสับสน และทำให้เกิดบักบ่อยที่สุดในโปรเจกต์ที่ใช้ GetX

กฎการแยก state — ตรวจสอบแล้วว่าทำถูกทุกจุด

ใช้	เมื่อไหร่ / ตัวอย่างจริงที่ตรวจสอบแล้ว
StatefulWidget + setState()	state เป็น ephemeral UI state ที่ไม่มีใครนอก widget สนใจ — ตรวจสอบ 4 ไฟล์ที่ใช้ setState (DiscountDialog, OptionSelectionSheet, MenuFormPage, StaffPage dialog) ยืนยันว่าเป็น local form/dialog state ทั้งหมด ไม่มีจุดไหนใช้ผิด
GetxController + .obs / Obx()	state เป็น app/business state ที่หน้าอื่นต้องรู้ด้วย — ตะกร้า (CartController), รายการออเดอร์, สถานะล็อกอิน ฯลฯ ตรวจสอบแล้วว่า 100% ของ controller extends GetxController ถูกต้อง ไม่มีการผสมสอง pattern ในโมเดลเดียวกัน

🔗 บักรวมที่พบระหว่างพัฒนาโปรเจกต์ — แก้แล้ว

ห้ามอ่านค่า observable ใน itemBuilder/callback ที่อยู่ข้างใน Obx() ต้องอ่านที่ scope บนสุดของ Obx(() { ... }) เท่านั้น เพราะ GetX ติดตามเฉพาะ observable ที่ถูกอ่าน ตอน builder รับครั้งแรก — ถ้าอ่านใน callback ที่เรียกทีหลัง (เช่นตอน scroll) GetX จะไม่รู้ว่าต้อง subscribe

```
// ❌ ผิด — chip กรองสถานะไม่ rebuild เมื่อกดเปลี่ยนตัวกรอง (บักรวมที่เจอใน orders_page.dart)
Obx(() => ListView.builder(
  itemBuilder: (context, i) {
    final selected = controller.statusFilter.value == filters[i].value; // ไม่ rebuild!
    return ChoiceChip(selected: selected, ...);
  },
));

// ✅ ถูก — อ่านค่าที่ scope บนสุดของ Obx ก่อน แล้วส่งต่อเข้า itemBuilder
Obx(() {
  final activeFilter = controller.statusFilter.value; // อ่านตรงนี้ทีเดียว
  return ListView.builder(
    itemBuilder: (context, i) {
      final selected = activeFilter == filters[i].value;
      return ChoiceChip(selected: selected, ...);
    },
  );
});
```

ไฟล์จริงที่แก้แล้ว: app/lib/features/order/presentation/pages/orders_page.dart

Dependency Injection และ Controller Lifecycle

- Composition root มีทีเดียว: app/lib/app/di/initial_binding.dart — ตรวจสอบแล้วที่ไม่มีการ ประกาศ Get.lazyPut/Get.put กระจัดกระจายนอกไฟล์นี้หรือ *_binding.dart
- Controller ที่มี StreamSubscription/Timer/TextEditingController ต้อง override onClose() — ตรวจสอบ 4 controller ที่เข้าเงื่อนไข (Auth, Checkout, Settings, Menu) ทำถูกครบทั้งหมด ไม่มี memory leak

**จุดที่ควรปรับปรุง (ไม่ใช่บั๊ก)** — พบ 10 จุดที่ private widget ย่อย (เช่น _TableSummaryBar, _UserChip) เรียก Get.find<T>() เองข้างในแทนที่จะรับ controller ผ่าน constructor หรือ extends GetView<T> — ผูก widget กับ service locator โดยตรง ทดสอบแยก ยากขึ้น บักรวมเป็นกฎในมาตรฐานให้รับเมื่อแตะไฟล์เหล่านี้ครั้งถัดไป

Clean Architecture

ตรวจสอบทิศทาง dependency ทุกไฟล์ใน domain layer และ backend layer ด้วยการใส่ import จริง ไม่ใช่แค่ดูโครงสร้างไฟล์เดอร์ผิวเผิน

presentation → **domain** ← **data**

domain ต้องเป็น pure Dart ไม่รู้จัก package:get, HTTP, หรือ widget ใดๆ

routes → **controller** → **service** → **repository** → **db**

service ห้ามรู้จัก req/res ของ Express

ผลตรวจ Backend — 9/9 module ไม่มี violation

เช็คว่า	ผลลัพธ์
service import req/res ของ Express	ไม่พบ
controller/routes import repository หรือ better-sqlite3 ขึ้น service	ไม่พบ

🐡 ผลตรวจ Flutter — พบ 2 จุด แก้แล้ว

พบว่า menu_repository.dart และ order_repository.dart ใน domain/repositories/ import MenuItemPayload จาก data/models/ และ OrderItemPayload จาก data/datasources/ โดยตรง — ผิดกฎทิศทาง dependency ตรงๆ

```
// ❌ ก่อนแก้ — domain/repositories/menu_repository.dart
import '../././././core/usecases/result.dart';
import '.././data/models/menu_item_model.dart'; // domain พึ่ง data!
import '../entities/category.dart';

// ✅ หลังแก้ — ย้าย MenuItemPayload ไปเป็น domain entity แล้ว
import '../././././core/usecases/result.dart';
import '../entities/menu_item_payload.dart'; // data → domain (ทิศทางถูก)
import '../entities/category.dart';
```

คลาส	ย้ายจาก	ย้ายไป
MenuItemPayload	features/menu/data/models/menu_item_model.dart	features/menu/domain/entities/menu_item_payload.dart
OrderItemPayload	features/order/data/datasources/order_remote_data_source.dart	features/order/domain/entities/order_item_payload.dart

ทำไมถึงผิด แม้คลาสจะมีแค่ field ธรรมดา — พารามิเตอร์ input ของ usecase/repository (ที่ส่งท้ายด้วย Payload/Params/Filter) ที่ไม่ได้ทำ HTTP เอง ต้องอยู่ใน domain/entities/ แม้จะมีเมธอด toJson() ติดไปด้วยก็ตาม เพราะ toJson() เป็นแค่ helper แปลงข้อมูล ไม่ใช่ transport-layer dependency — สิ่งที่ต้องสืบว่าคลาสอยู่ layer ไหนคือ **ตำแหน่งไฟล์และทิศทาง import** ไม่ใช่ว่ามันมีเมธอด serialize หรือเปล่า

ยืนยันหลังแก้: อัปเดต import ทุกจุดที่ใช้ (controller, page, repository impl, datasource, demo data source, test) — flutter analyze 0 ปัญหา, flutter test 46/46 ผ่าน เหมือนเดิม ไม่มีการเปลี่ยนพฤติกรรมใดๆ

Technical Debt

รายการที่ต้องติดตามต่อจากนี้ — ไม่ใช่สิ่งที่ต้องหยุดพัฒนาฟีเจอร์ใหม่รอแก้ก่อน แต่ถ้าแต่ละไฟล์ที่เกี่ยวข้องอยู่แล้วให้ถือโอกาสแก้ไปพร้อมกัน (boy scout rule)

#	รายการ	ผลกระทบ	สถานะ
1	domain layer import จาก data layer (MenuItemPayload, OrderItemPayload)	ผิดกฎ Clean Architecture โดยตรง — ย้ายเข้า domain/entities/ แล้ว	แก้แล้ว
2	3 backend module (payments/reports/settings) ไม่มี controller layer	ไม่สม่ำเสมอกับสถาปัตยกรรมที่ประกาศไว้ — เพิ่ม controller ให้ครบทั้ง 3 module แล้ว	แก้แล้ว
3	Backend ไม่มี ESLint/Prettier	style/simple bug ไม่ถูกจับอัตโนมัติ นอกจาก test coverage	ค้าง
4	Controller ส่วนใหญ่ใน Flutter ไม่มี unit test เฉพาะตัว (มีแค่ CartController)	บัก logic ใน controller จับได้ช้าลง ต้องพึ่ง manual QA	ค้าง
5	Backend module ส่วนใหญ่ไม่มี unit test เฉพาะ module	อาศัย integration test เดียวคุมทั้งระบบ ถ้า fail จะไม่รู้กันที่ว่าโมดูลไหนพัง	ค้าง
6	demo_store.dart 1,142 บรรทัดในไฟล์เดียว	แก้ยากขึ้นเรื่อยๆ เมื่อเพิ่ม demo scenario ใหม่	ค้าง
7	Flutter dependencies ล้าหลัง ~15 แพ็กเกจ (minor version)	ไม่กระทบการทำงาน แต่ควรตามให้ทันเป็นระยะ	ค้าง

Debt ที่ตั้งใจ — ไม่ต้องแก้ (กันสับสนกับของค้างจริง)

รายการเหล่านี้เป็นการตัดสินใจเชิงออกแบบที่มีเหตุผลรองรับแล้วใน docs/DECISIONS.md:

- ตรรกะคิดนิลเขียนซ้ำ 2 ภาษา (Dart preview + JS source of truth) — มีเหตุผลยืนยันด้วยตัวเลขชุดเดียวกันทั้งคู่
- สถานะ (OrderStatus, UserRole) เป็น string constants class ไม่ใช่ Dart enum จริง — เพราะ mirror กับ string enum ฝั่ง backend ตรงๆ ป้องกัน typo ด้วย named constants อยู่แล้ว (ตรวจแล้วว่าไม่มีจุดไหน compare ด้วย string literal ตรงๆ เลย)
- Result<T> เขียนเอง แทน dartz — compiler บังคับจัดการทั้งสองกรณีอยู่แล้ว

โครงสร้างโฟลเดอร์และตำแหน่งไฟล์

ตรวจสอบว่าไฟล์ทุกไฟล์อยู่ในตำแหน่งที่คนอื่นหาเจอโดยไม่ต้องเดา และทุก feature/module ใช้โครงสร้างเดียวกัน

Flutter — feature-first + Clean Architecture

ตรวจสอบทั้ง 9 feature (auth, home, kitchen, menu, order, payment, report, settings, staff, table) — ทุก feature ตามโครงสร้างเดียวกัน:

```
features/<feature>/
domain/
  entities/    โมเดลธุรกิจ ไม่ผูกกับ API shape
  repositories/ abstract class เท่านั้น
  usecases/    1 usecase = 1 class ที่มี call()
data/
  models/      extends entity + fromjson/tojson
  datasources/ เรียก ApiClient ตรงๆ
  repositories/ implements domain repository interface
presentation/
  bindings/    controllers/  pages/    widgets/
```

9/9 ผ่าน — ไม่บังคับว่าทุก feature ต้องมีโฟลเดอร์ widgets/ ถ้า page ไม่มี widget ที่ใหญ่พอจะแยก (เช่น staff/, settings/)

Backend — module-per-domain

```
modules/<module>/
<name>.routes.js   ผูก path + middleware
<name>.controller.js แปลง req → เรียก service → ส่ง response
<name>.service.js   business logic ล้วนๆ
<name>.repository.js SQL query
<name>.mapper.js    แปลง DB row ↔ API shape
<name>.schema.js    zod validation
```

ข้อไม่สม่ำเสมอที่พบและแก้แล้ว

payments/, reports/, settings/ เคยไม่มีไฟล์ .controller.js — route handler เรียก xxxService.method() ตรงจาก routes.js เลย ในขณะที่อีก 6 module มี controller คั่นกลางตามสถาปัตยกรรมที่ประกาศไว้

Module	ก่อนแก้	หลังแก้
payments	route → service ตรง	route → controller → service
reports	route → service ตรง	route → controller → service
settings	route → service ตรง	route → controller → service

ยืนยันหลังแก้: เพิ่ม payment.controller.js, report.controller.js, settings.controller.js ตามรูปแบบเดียวกับโมดูลที่มีอยู่แล้ว (object literal ห่อ asyncHandler ดัด method) path/middleware/response shape เต็มทุกอย่าง — npm test 36/36 ผ่านเหมือนเดิม และยัง API จริงตรวจ 3 endpoint ผ่าน คน (GET /settings, GET /reports/dashboard, GET /payments/order/:id)

9/9 ผ่าน — ตอนนี้ backend ทั้ง 9 module มี routes → controller → service → repository คนเหมือนกันหมดแล้ว ไม่มีการยกเว้นเหลืออยู่

สถานะหลังการตรวจและแก้ไข

มิติ	สถานะสุดท้าย
Clean Code	ดี
State Management (GetX)	ดี
Clean Architecture	แก้ครบแล้ว
Technical Debt	ติดตาม 5 รายการ
โครงสร้างไฟล์เดอร์	แก้ครบแล้ว

Checklist ก่อน commit / เปิด PR จากนั้นไป

```
# Flutter
cd app
flutter analyze
dart format --output=none --set-exit-if-changed .
flutter test

# Backend
cd backend
npm test
```

 รายละเอียดทุกทั้งหมัด ตัวอย่างโค้ด ก่อน/หลัง และคำสั่ง grep สำหรับเช็ค layer violation ก่อน commit — ดูฉบับเต็มที่มีชีวิตและอัปเดตต่อเนื่องได้ที่ [docs/CODING_STANDARDS.md](#) ในโปรเจกต์

รายงานนี้สร้างจากผลตรวจจริง (flutter analyze, dart format, flutter test, npm test, การไล่ import ที่ละไฟล์) ไม่ใช่การประเมินเชิงทฤษฎี — ทุกข้อค้นพบยืนยันขึ้นด้วยการรันจริง ก่อนสรุปผล